

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Artificial Intelligence and Data Science
ASSESSMENT TOOL 2 – Pseudocode Writing Task (Algorithm Design)

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO5 – Articulation	Assessment:	Assessment Tool 2 – Pseudocode Writing Task (Algorithm Design)
Weightage:	35% of CIA		
CO5 Statement:	Implement semantic models using predicate logic, word sense disambiguation and validate information retrieval techniques. (BL-5)		
Student Name:	M Poojitha Reddy	Reg. No.:	192472352
Section / Batch:	2024	Date:	3-09-26

Code of Conduct

I, M Poojitha Reddy - 192472352 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: M Poojitha Reddy

Instructions

- Draw necessary diagrams such as constraint graphs, coreference chains, discourse trees, or predicate logic representations wherever applicable.
- Generate solutions that fully satisfy all given constraints and provide clear evaluation of the output.
- Answers should be concise, well-structured, and supported by evidence from the given text/scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Reference Resolution, Discourse	Entity Linking Algorithm	Q1
Text Coherence, Discourse Structure	Coherence Analysis Algorithm	Q2
Dialog Acts, Conversational Agents	Intent Recognition Algorithm	Q3
Language Generation, Surface Realization	NLG Pipeline Design	Q4
Machine Translation, Interlingua, Statistical Approaches	Translation Workflow Algorithm	Q5

Annexure A – Pseudocode Writing Task (Algorithm Design)

1. Reference Resolution Using Multiple Constraints

Design an algorithm to perform **Reference Resolution** by identifying pronouns and selecting the most appropriate antecedent using gender, number, recency, and semantic compatibility.

Apply your algorithm to the discourse:

"Meena gave Kavya a laptop because she needed one for her project. She thanked her and started using it."

Perform the following steps:

1. Identify all entities and referring expressions.
2. Generate possible antecedents for each pronoun.
3. Apply gender, number, recency, and semantic constraints.
4. Select the most appropriate antecedent.
5. Generate the resolved discourse.

Finally, explain how your algorithm avoids incorrect resolution when multiple antecedents have similar grammatical properties.

2. Algorithm for Entity Chains and Discourse Coherence

Design an algorithm to identify **entity chains** in a discourse and use them to evaluate text coherence.

Apply your algorithm to the following text:

"Anjali bought a new laptop. The laptop had a powerful processor. She installed Python on it. Later, Anjali used the computer to develop a machine learning application."

Perform the following steps:

1. Identify all entities and referring expressions.
2. Link expressions referring to the same entity.
3. Construct entity chains.
4. Calculate or estimate the continuity of entities across sentences.
5. Determine whether the discourse is coherent.

Finally, explain how breaking an entity chain could reduce discourse coherence.

3. Algorithm for Identifying Discourse Relations

Design an algorithm to identify **discourse relations** between consecutive sentences using discourse markers and contextual meaning.

Apply your algorithm to:

"The server crashed unexpectedly. As a result, users could not access the application. The technical team restarted the server. After that, the system became available again."

Your algorithm should identify relations such as:

- Cause–Effect
- Sequence
- Problem–Solution
- Elaboration

Perform the following steps:

1. Divide the discourse into individual discourse units.
2. Identify explicit discourse markers.
3. Analyze the semantic relationship between units.

4. Assign an appropriate discourse relation.
5. Construct a discourse structure representation.

Finally, justify how the identified relations make the discourse logically coherent.

4. Algorithm for Dialogue Act Classification

Design an algorithm to recognize **Dialogue Acts** in a conversation.

Apply your algorithm to:

User: "Hello, I need help with my assignment."

Agent: "Sure, what problem are you facing?"

User: "I do not understand machine learning."

Agent: "I can explain the basic concepts to you."

Classify each utterance into one of the following:

- Greeting
- Request
- Question
- Inform
- Offer
- Confirmation

Perform the following steps:

1. Preprocess each utterance.
2. Extract relevant linguistic features.
3. Identify the communicative intention.
4. Assign a dialogue act.
5. Generate the dialogue-act sequence.

Finally, explain how dialogue-act recognition helps a conversational agent select an appropriate next response.

5. Algorithm for Dialogue State Tracking

Design an algorithm for a **Conversational Agent** that maintains dialogue state while collecting information from a user.

Apply your algorithm to:

User: "I want to book a flight."

Agent: "Where would you like to travel?"

User: "I want to go to Delhi."

Agent: "From which city?"

User: "From Chennai."

Your algorithm should maintain the following slots:

Departure City

Destination City

Travel Date

Number of Passengers

Perform the following steps:

1. Initialize the dialogue state.
2. Identify information provided in each utterance.
3. Update the corresponding slot.
4. Identify missing information.

5. Generate the next appropriate question.
Finally, explain how dialogue state tracking improves coherence in a multi-turn conversation.

Rubrics for Calculation

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Q. No. / Criteria Level	Problem Understanding	Algorithm Design	Use of Control Structures	Pseudocode Structure	Completeness	Sub. Total	Total %
1	4	4	4	3	4	19	97
2	4	4	4	4	4	20	
3	4	4	4	4	4	20	
4	4	4	4	3	4	19	
5	4	3	4	4	4	19	

Faculty In-charge	Course Coordinator	Program Director
T Kumaragurubaran		
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1. Reference Resolution Using Multiple Constraints

Discourse:

“Meena gave Kavya a laptop because she needed one for her project. She thanked her and started using it.”

Entities:

- Meena – female, singular, person
- Kavya – female, singular, person
- Laptop – singular, object
- Project – singular, object
-

Referring expressions: she, one, her, She, her, it

Algorithm:

1. Identify pronouns and referring expressions.
2. Find possible antecedents from previously mentioned entities.
3. Check gender and number compatibility.
4. Check semantic compatibility.
5. Use recency and grammatical role as additional factors.
6. Select the candidate with the highest score.

Pseudocode:

BEGIN

 Identify entities and pronouns

 FOR each pronoun

 Find possible antecedents

 FOR each candidate

 Check gender

 Check number

 Check semantic compatibility

 Check recency

 Calculate score

 END FOR

 Select candidate with highest score

 END FOR

Output resolved discourse
END

Resolution:

Expression	Antecedent
she	Kavya
one	laptop
her (project)	Kavya
She	Kavya
her (thanked)	Meena
it	laptop

Resolved discourse:

“Meena gave Kavya a laptop because Kavya needed a laptop for her project. Kavya thanked Meena and started using the laptop.”

The algorithm avoids incorrect resolution because it does not depend only on recency. Gender, number, semantic compatibility and grammatical context are considered together. This is especially important because both Meena and Kavya are female and singular.

2. Algorithm for Entity Chains and Discourse Coherence

Discourse:

“Anjali bought a new laptop. The laptop had a powerful processor. She installed Python on it. Later, Anjali used the computer to develop a machine learning application.”

Entities and referring expressions:

- Anjali
- laptop
- processor
- Python
- application
- She → Anjali
- it → laptop
- Anjali → Anjali
- computer → laptop

Entity chains:

- **Anjali:** Anjali → She → Anjali
- **Laptop:** laptop → The laptop → it → the computer
- **Processor:** processor
- **Python:** Python
- **Application:** machine learning application

Algorithm:

1. Divide the text into sentences.
2. Extract noun phrases and pronouns.
3. Compare each expression with previously mentioned entities.
4. Link expressions referring to the same entity.
5. Construct entity chains.
6. Calculate how frequently entities continue across sentences.
7. Use continuity to estimate coherence.

Pseudocode:

BEGIN

Split discourse into sentences

Extract entities and referring expressions

FOR each expression

Find matching previous entities

IF matching entity exists

Add expression to its entity chain

ELSE

Create a new entity chain

END IF

END FOR

Calculate entity continuity

Evaluate discourse coherence

END

Continuity:

- Anjali appears across 3 of 4 sentences → **0.75**
- Laptop appears or is referred to across all 4 sentences → **1.00**

Therefore, the discourse is **coherent** because the main entities are continuously maintained.

If an entity chain is suddenly broken without explanation, the reader may find the text confusing.

For example, suddenly replacing the laptop with an unrelated object would reduce topic continuity and coherence.

3. Algorithm for Identifying Discourse Relations

Discourse:

“The server crashed unexpectedly. As a result, users could not access the application. The technical team restarted the server. After that, the system became available again.”

Discourse Units:

- DU1: Server crashed unexpectedly.
- DU2: Users could not access the application.
- DU3: Technical team restarted the server.
- DU4: System became available again.

Discourse markers:

- “As a result” → Cause–Effect
- “After that” → Sequence

Relations:

- DU1 → DU2 = **Cause–Effect**
- DU2 → DU3 = **Problem–Solution**
- DU3 → DU4 = **Sequence/Result**

Pseudocode:

BEGIN

 Split discourse into discourse units

 FOR each consecutive pair of units

 Identify discourse markers

 IF cause marker exists

 Relation = Cause-Effect

 ELSE IF sequence marker exists

 Relation = Sequence

 ELSE IF first unit is a problem

 AND second unit is a solution

 Relation = Problem-Solution

 ELSE

 Relation = Elaboration

 END IF

 END FOR

Output discourse structure
END

Discourse structure:

Server crashed



Cause-Effect



Users could not access application



Problem-Solution



Server restarted



Sequence/Result



System became available

These relations make the discourse coherent because the events follow a logical **problem** → **consequence** → **solution** → **result** structure.

4. Algorithm for Dialogue Act Classification

Conversation:

User: "Hello, I need help with my assignment."

Agent: "Sure, what problem are you facing?"

User: "I do not understand machine learning."

Agent: "I can explain the basic concepts to you."

Algorithm:

1. Preprocess each utterance.
2. Extract keywords and sentence patterns.
3. Identify the communicative intention.
4. Assign a dialogue act.

Pseudocode:

BEGIN

FOR each utterance

Preprocess utterance

Extract linguistic features

```
IF greeting pattern
  Act = Greeting
ELSE IF question pattern
  Act = Question
ELSE IF request pattern
  Act = Request
ELSE IF offer pattern
  Act = Offer
ELSE IF confirmation pattern
  Act = Confirmation
ELSE
  Act = Inform
END IF
END FOR
END
```

Classification:

Speaker	Utterance	Dialogue Act
User	Hello, I need help with my assignment.	Request
Agent	Sure, what problem are you facing?	Question
User	I do not understand machine learning.	Inform
Agent	I can explain the basic concepts to you.	Offer

Dialogue-act sequence:

Request → Question → Inform → Offer

Dialogue-act recognition helps a conversational agent understand the user's intention and choose an appropriate response. For example, after detecting an **Inform** act, the agent can provide an explanation or solution.

5. Algorithm for Dialogue State Tracking

Required slots:

- Departure City
- Destination City
- Travel Date
- Number of Passengers

Initial state:

Departure City = UNKNOWN

Destination City = UNKNOWN

Travel Date = UNKNOWN

Number of Passengers = UNKNOWN

Algorithm:

1. Initialize all slots as unknown.
2. Process each user utterance.
3. Extract the information provided.
4. Update the appropriate slot.
5. Find missing slots.
6. Ask a question for the next missing slot.

Pseudocode:

BEGIN

 Initialize all slots as UNKNOWN

 FOR each user utterance

 Extract information

 IF destination is found

 Update Destination City

 END IF

 IF departure city is found

 Update Departure City

 END IF

 IF travel date is found

 Update Travel Date

 END IF

IF passenger count is found
 Update Number of Passengers
END IF

Find missing slots
Ask question for the next missing slot
END FOR
END

Step-by-step state updates:

After:

User: “I want to book a flight.”

Departure = UNKNOWN

Destination = UNKNOWN

Date = UNKNOWN

Passengers = UNKNOWN

Agent asks: **“Where would you like to travel?”**

After:

User: “I want to go to Delhi.”

Departure = UNKNOWN

Destination = Delhi

Date = UNKNOWN

Passengers = UNKNOWN

Agent asks: **“From which city?”**

After:

User: “From Chennai.”

Departure = Chennai

Destination = Delhi

Date = UNKNOWN

Passengers = UNKNOWN

The next appropriate question is:

“What is your travel date?”

Dialogue state table:

Turn	Departure	Destination	Date	Passengers
Initial	Unknown	Unknown	Unknown	Unknown
Go to Delhi	Unknown	Delhi	Unknown	Unknown
From Chennai	Chennai	Delhi	Unknown	Unknown

Dialogue State Tracking improves coherence because the agent remembers information from earlier turns and does not repeatedly ask for information that the user has already provided.